The timetable algorithm

A step-by-step guide to the scheduling logic in this project.

The weekly timetable is a constraint-satisfaction problem. The app turns each lecture or lab into a graph vertex, assigns it a day-and-time slot, and rejects choices that break class, faculty, lab-day, or room-capacity rules.

1. Create the sessions

For every entered subject, create a lecture session for EXCS A and another for EXCS B. If Has lab is enabled, create one lab session for each class too. A session carries its subject, class, and type.

Before searching, split the room list into lecture rooms and lab rooms. A room whose name starts with uppercase L is treated as a lab room; all other names are lecture rooms. If a required room pool is empty, scheduling stops as impossible.

2. Build the conflict graph

Treat each session as a vertex. Compare every pair of sessions. Add an undirected edge when both belong to the same class, because a class cannot attend two sessions at once; also add an edge when their faculty names match after trimming whitespace and ignoring letter case.

The graph records which sessions cannot share an exact slot. Room limits are handled separately as a capacity rule, rather than by adding graph edges. For each subject and class, remember its lecture/lab pair so their days can be kept apart.

3. Define the possible colors (slots)

A graph color is a day-and-time pair, not a room or a visual subject color. There are 5 weekdays and 4 periods per day, so the search has 20 colors. Slot number s maps to day floor(s / 4) and period s modulo 4.

A candidate slot is valid only if no already-assigned graph neighbor uses it, the same subject class does not already have its lecture or lab on that day, and fewer same-type sessions than rooms are already using that slot.

Choosing and testing time slots

DSatur guides the search; randomized ordering helps vary valid schedules.

4. Pick the next session with DSatur

Among all unassigned sessions, count the distinct slots already used by each session assigned neighbors. This number is its saturation. Select a session with the greatest saturation: it is the most constrained by choices already made.

If saturation ties, prefer the session with the most still-unassigned neighbors (its remaining degree). If that also ties, choose randomly among the tied sessions. This is the DSatur-style ordering used to reduce avoidable dead ends.

5. Order candidate slots to spread a class weekly workload

Start with all 20 slots in Fisher-Yates shuffled order, then discard slots that fail the constraints. Sort the remaining candidates by how many sessions that class already has on each day. The least-loaded days come first; the earlier shuffled order breaks equal-load ties.

This balancing preference is a heuristic, not a hard rule. It encourages a more even weekly distribution while still allowing the search to use a busier day when necessary.

6. Apply the constraints for a candidate

Conflict check: every neighboring session that is already assigned must have a different slot. The same rule covers the class and faculty conflicts encoded in the graph.

Lab-day check: when the paired lecture or lab is already placed, reject any slot on that same weekday. Room-capacity check: count assigned sessions of this session type in this exact slot and require the count to be smaller than the number of rooms of that type.

Lecture and lab room pools are independent. For example, a lecture and a lab may both use the same time if their respective room pools have enough rooms.

Backtracking and room assignment

A failed branch is undone exactly, then the search tries another choice.

7. Search recursively and backtrack

Step 1 - If every session is assigned, report success. Otherwise choose the next session with DSatur and prepare its valid candidate slots.

Step 2 - Try a candidate: store its slot and increase that class day load. Recursively attempt to place the remaining sessions.

Step 3 - If the recursive call succeeds, keep the assignment. If it fails, undo both changes (decrease the day load and mark the session unassigned), then try the next candidate.

Step 4 - If every candidate fails, return failure to the previous level. That level also undoes its choice and explores its next candidate. If all branches fail, the app reports that no schedule was found.

A quick impossibility check rejects a class with more than 20 sessions, and missing required room pools are rejected before search. Graph coloring with backtracking can take exponential time in the worst case; DSatur and constraint checks are practical search heuristics, not a guarantee of fast completion.

8. Allocate actual rooms after times fit

Once every session has a slot, process the sessions and track the rooms already used for each slot-and-session-type pair. Shuffle that type room list, choose one not yet used in that pair, and mark it used.

This gives simultaneous lectures different lecture rooms and simultaneous labs different lab rooms. The earlier capacity test guarantees enough rooms for each pair; an unexpected shortage raises an explicit error.

9. Show class and teacher views

The class view filters the generated sessions to EXCS A or EXCS B. The teacher view filters the same results to the selected faculty member, so it combines that teachers sessions across both classes. These views do not run a second scheduling algorithm.

Exams and randomization

The exam planner is separate from weekly graph coloring.

10. Place one exam per subject

Start with the UTC dates November 16 and December 16, 2026. If even minimum two-day gaps between all exams would exceed this window, stop with an explanatory error: with n subjects the minimum span is (n - 1) times 2 days.

Shuffle the subject list, then place the first subject on November 16. For each later subject, calculate how much gap remains possible while reserving at least two days for every later transition. If a three-day gap still fits, randomly add either 2 or 3 days; otherwise add 2 days.

A two-day date difference leaves one free calendar day between exams; a three-day difference leaves two. Reserving the minimum for the remaining exams keeps every date on or before December 16. The algorithm does not exclude weekends.

11. How the random shuffle works

The shared shuffled helper copies its input so the original list is unchanged. Starting at the last position and moving left, it picks a random position from zero through the current position and swaps those two items. This is the Fisher-Yates shuffle.

That helper randomizes ties between equally ranked sessions, candidate-slot tie order, room choice order, and exam subject order. Pressing Generate again can therefore produce a different valid timetable and exam order. The random generator is not seeded, so the same inputs do not promise the same output.

12. What is guaranteed - and what is preferred

Hard rules: class/faculty slot conflicts, enough rooms at each slot, and different days for a subjects lecture and lab. Exam dates stay in the stated window with one or two free days between consecutive exams. Preference: spread each class sessions across less-loaded weekdays when possible.

The schedule search can fail when the supplied subjects and rooms cannot meet the rules. Adding rooms or reducing subjects may make a solution possible. A new randomized attempt can also explore a different branch order, but does not change the underlying constraints.